

Lab 12.1 - Observe ol-lab-01 Under Load

KVM Virtualisation on Oracle Linux - Day 2, Module 12 (Monitoring & Observability)

Overview

Every lab so far has checked a VM's state at rest - running, shut off, correctly configured. This lab watches ol-lab-01 while it's actually busy: you'll capture a quiet baseline, generate real CPU and disk load inside the guest, and watch virsh domstats and virt-top's numbers move live on the host. You'll finish by streaming ol-lab-01's lifecycle events in real time and confirming they fire exactly when expected - the same event stream a monitoring or alerting system would hook into in production.

This lab assumes ol-lab-01 exists and is running, with the /data disk from Lab 5.1 available inside the guest.

Note: This lab uses three terminal sessions at once - one connected to ol-lab-01 itself (to generate load), and two separate sessions on the host (one for domstats/virt-top, a second kept free for the live event stream so it isn't blocked by anything else). Open all three before starting Step 1.

Learning Objective

By the end of this lab, you will be able to:

- Capture a VM's baseline resource usage with virsh domstats and virt-top
- Generate real CPU and disk load inside a guest and observe it reflected live on the host
- Stream a VM's lifecycle events in real time with virsh event
- Confirm that stop/start actions produce the events a monitoring system would expect to see

Step by Step Guide

Prerequisite: Connect and Open Your Terminals

1. Terminal 1 - connect directly to ol-lab-01 itself, since this is where you'll generate load:

```
sudo virsh domifaddr ol-lab-01  
ssh -i ~/.ssh/id_ed25519 cloud-user@<ol-lab-01's address>
```
2. Terminal 2 - a separate session on the host, for baseline checks and later the stop/start commands:

```
ssh opc@<your-lab-instance-ip>
```
3. Terminal 3 - another separate session on the host, kept free specifically for the live event stream in Step 3:

```
ssh opc@<your-lab-instance-ip>
```

Expected result: Three active terminal sessions: one inside ol-lab-01, two on the host.

Step 1: Baseline with virsh domstats and virt-top

Run these in Terminal 2, before any load has been generated, so you have a genuine quiet baseline to compare against later.

4. Capture a structured snapshot of ol-lab-01's current stats:

```
sudo virsh domstats ol-lab-01
```

Expected result: A block of cpu.time, balloon (memory), and block (disk) statistics. Note the cpu.time value specifically - you'll watch this climb in Step 2.

5. Install virt-top if it isn't already present, then view the live per-VM dashboard:

```
sudo dnf install -y virt-top
```

```
sudo virt-top
```

Expected result: A live-updating table of every VM on the host, refreshing every few seconds. *ol-lab-01* should show close to 0% CPU, since nothing is generating load yet.

6. Press q to exit virt-top once you've noted the quiet baseline - Terminal 2 is needed again shortly.

Step 2: Generate Load and Watch the Numbers Move

Now generate real CPU and disk activity inside the guest, and watch it reflected live on the host.

7. In Terminal 1 (inside *ol-lab-01*), install a load-generation tool:

```
sudo dnf install -y stress-ng
```

8. Start a CPU load matching *ol-lab-01*'s 2 vCPUs, running in the background for two minutes:

```
stress-ng --cpu 2 --timeout 120s &
```

9. Also generate disk load, writing to the /data disk from Lab 5.1:

```
dd if=/dev/zero of=/data/loadtest.img bs=1M count=2000 oflag=direct
```

Expected result: *dd* reports its transfer rate once complete. While it and *stress-ng* are running, *ol-lab-01*'s CPUs are genuinely busy, not just idling.

10. Back in Terminal 2 on the host, re-run the same two baseline commands from Step 1:

```
sudo virsh domstats ol-lab-01
```

```
sudo virt-top
```

Expected result: *domstats*' *cpu.time* value has climbed sharply compared to the Step 1 baseline. *virt-top* shows *ol-lab-01*'s %CPU column now reading genuinely high, close to what 2 fully busy vCPUs would show - live confirmation the host-side view directly reflects what's happening inside the guest, not a cached or delayed figure.

11. Press q to exit virt-top once you've confirmed the load is visible, then clean up the test file once *stress-ng* and *dd* have finished:

```
rm -f /data/loadtest.img
```

Step 3: Stream Live Lifecycle Events

In Terminal 3 - the session kept free for exactly this - start streaming every libvirt event across every VM on the host, in real time.

12. Start the event stream:

```
sudo virsh event --loop --all
```

Expected result: The command does not return - it blocks, waiting to print each event as it happens. Leave this running and switch to Terminal 2 for the next step.

Note: *--all* subscribes to every event type (lifecycle, reboot, disk, and more) across every domain on the host, not just *ol-lab-01* - in a real monitoring setup, this is the same stream a script would parse to trigger alerts or automation.

Step 4: Stop and Start the VM - Confirm the Lifecycle Events Fire

13. In Terminal 2, shut *ol-lab-01* down:

```
sudo virsh shutdown ol-lab-01
```

Expected result: Switching to Terminal 3, a lifecycle event appears for *ol-lab-01* showing a *Stopped* reason almost immediately once the guest OS finishes shutting down - the event stream reacts in real time, not on a polling delay.

14. Confirm it's genuinely off, then start it again from Terminal 2:

```
sudo virsh list --all
```

```
sudo virsh start ol-lab-01
```

Expected result: *Terminal 3 shows a second lifecycle event for ol-lab-01, this time showing a Started reason - confirmation the stream reliably captures both directions of the lifecycle, not just shutdowns.*

15. Once both events have been observed, stop the stream with Ctrl-C in Terminal 3.

Outcomes

By the end of this lab, you have captured a genuine baseline for ol-lab-01, watched real CPU and disk load reflected live through virsh domstats and virt-top, and confirmed that stopping and starting a VM produces lifecycle events you can observe in real time. This is the same observability foundation a production monitoring or alerting system is built on - not simulated or delayed data, but the host reacting to exactly what's happening inside the guest, and reporting it as it happens.